Exercicio 4

Eduardo Rios de Andrade Sousa – 2022087105

Questão 1)

Código:

```
import numpy as np

import matplotlib.pyplot as plt

from mpl_toolkits.mplot3d import Axes3D


# 1. Definindo os parâmetros

n_amostras = 200


covariancia = [[0.16, 0],

               [0, 0.16]]


# 2. Gerando os dados da Classe 1

media_1 = [2, 2]

X1 = np.random.multivariate_normal(media_1, covariancia, n_amostras)


# 3. Gerando os dados da Classe 2

media_2 = [4, 4]

X2 = np.random.multivariate_normal(media_2, covariancia, n_amostras)


# 4. Visualizando os dados (Plot)

plt.figure(figsize=(8, 6))


# Plotando com círculos vazados para ficar parecido com o R

plt.scatter(X1[:, 0], X1[:, 1], facecolors='none', edgecolors='red', label='Classe 1
(Média 2,2)')
```

```
plt.scatter(X2[:, 0], X2[:, 1], facecolors='none', edgecolors='blue', label='Classe 2 (Média 4,4)')

# Ajustes do gráfico para bater com os eixos do exercício original
plt.xlim(-2, 8)
plt.ylim(-2, 8)
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.legend()
plt.title('Dados Amostrados - Distribuições Normais')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

# --- 1. Preparando os dados ---

# Juntando as matrizes X1 e X2 em um único conjunto de dados
X_dados = np.vstack((X1, X2))

# Adicionando a coluna do Bias (coluna de 1s no início)
bias = np.ones((X_dados.shape[0], 1))
X_com_bias = np.hstack((bias, X_dados))

# Criando os rótulos (y): 200 zeros para a Classe 1, 200 uns para a Classe 2
y_dados = np.hstack((np.zeros(n_amostras), np.ones(n_amostras)))

# Embaralhando os dados (boa prática em IA para o modelo não "decorar" a ordem)
indices = np.arange(X_com_bias.shape[0])
np.random.shuffle(indices)
X_treino = X_com_bias[indices]
```

```
y_treino = y_dados[indices]

# --- 2. Implementando o Perceptron ---

def trainperceptron(X, y, eta=0.1, max_epochs=100):
    """
    Treina um Perceptron Simples.
    eta: Taxa de aprendizado (learning rate)
    max_epochs: Limite máximo de iterações sobre o dataset
    """
    n_amostras, n_features = X.shape

    # Inicializando o vetor de pesos (w) com zeros
    # Como adicionamos o bias no X, w terá tamanho 3: [w0, w1, w2]
    w = np.zeros(n_features)

    for epoca in range(max_epochs):
        erro_total = 0

        for i in range(n_amostras):
            # Passo A: Calcula a soma ponderada (u = X * w)
            u = np.dot(X[i], w)

            # Passo B: Aplica a função de ativação (Degrau)
            y_pred = 1 if u >= 0 else 0

            # Passo C: Calcula o erro
            erro = y[i] - y_pred
```

```
        # Passo D: Atualiza os pesos (Regra de Hebb/Perceptron)
        if erro != 0:
            w = w + eta * erro * X[i]
            erro_total += 1

    return w

# --- 3. Executando o treinamento ---

w_final = trainperceptron(X_treino, y_treino)
print("Vetor de pesos encontrado (w):", w_final)

# --- 1. Preparação para os Gráficos ---
# Cria uma figura com espaço para dois gráficos (1 linha, 2 colunas)
fig = plt.figure(figsize=(14, 6))

# --- 2. Gráfico 2D: Dispersão e Fronteira de Decisão ---
ax1 = fig.add_subplot(1, 2, 1)

# Plotando os dados originais
ax1.scatter(X1[:, 0], X1[:, 1], facecolors='none', edgecolors='red', label='Classe 1')
ax1.scatter(X2[:, 0], X2[:, 1], facecolors='none', edgecolors='blue', label='Classe 2')

# Criando a reta de decisão baseada nos pesos (w_final) que você treinou
x1_reta = np.linspace(0, 6, 100)
# Aplicação da fórmula: x2 = -(w0 + w1*x1) / w2
x2_reta = -(w_final[0] + w_final[1] * x1_reta) / w_final[2]

ax1.plot(x1_reta, x2_reta, color='black', linewidth=3, label='Fronteira')
```

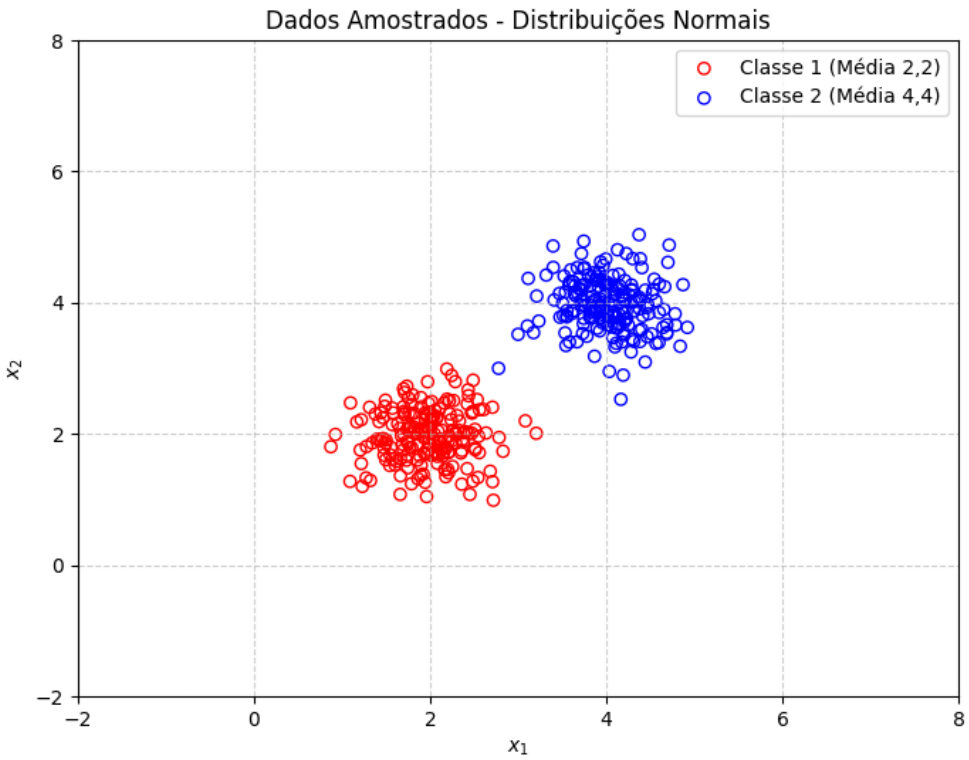
```
ax1.set_xlim(0, 6)
ax1.set_ylim(0, 6)
ax1.set_xlabel('$x_1$')
ax1.set_ylabel('$x_2$')
ax1.legend()
ax1.set_title('Classificação 2D')
ax1.grid(True, linestyle='--', alpha=0.6)

# --- 3. Gráfico 3D: Superfície de Resposta ---
ax2 = fig.add_subplot(1, 2, 2, projection='3d')

# Criando uma malha de coordenadas (grid) de 0 a 6 com passo de 0.1
seqi = np.arange(0, 6.1, 0.1)
seqj = np.arange(0, 6.1, 0.1)
X1_mesh, X2_mesh = np.meshgrid(seqi, seqj)

# Calculando a ativação para todos os pontos da malha simultaneamente
U = w_final[0] + w_final[1] * X1_mesh + w_final[2] * X2_mesh

# Aplicando a função degrau: se U >= 0, recebe 1 (azul), senão 0 (vermelho)
M = np.where(U >= 0, 1, 0)

# Plotando a superfície (cmap='jet' gera o padrão de cores do azul pro vermelho)
surf = ax2.plot_surface(X1_mesh, X2_mesh, M, cmap='jet_r', edgecolor='none',
alpha=0.9)

ax2.set_xlabel('$x_1$')
ax2.set_ylabel('$x_2$')
ax2.set_zlabel('Saída do Perceptron')
ax2.set_title('Superfície de Decisão 3D')
```

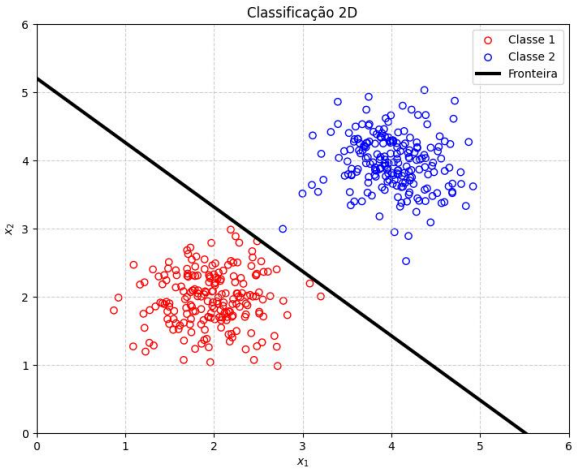
```
# Ajustando o ângulo de visão da câmera 3D para ficar parecido com o PDF
ax2.view_init(elev=35, azim=-125)

# Adicionando a barra de cores lateral
fig.colorbar(surf, ax=ax2, shrink=0.5, aspect=10)

plt.tight_layout()
plt.show()
```

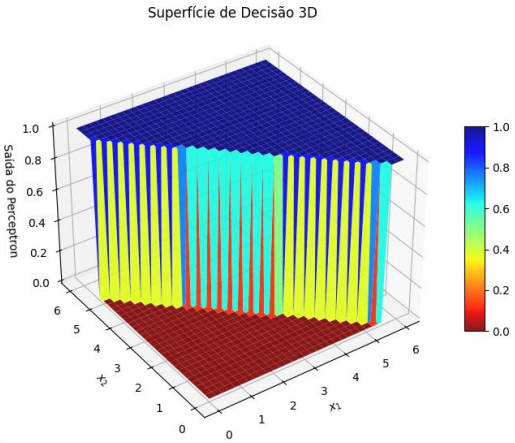
OBS: No pdf da lista, também foram utilizadas variâncias de 0.16 para cada variável.

Graficos e resultados:

```
Vetor de pesos encontrado (w): [-2.1        0.38074171  0.40298885]
```







Questão 2)

Código:

```
import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, confusion_matrix


# ==========================================

# 1. GERAÇÃO DOS DADOS (Conforme Exercício 1 e 2)

# ==========================================

n_amostras = 200 # 200 amostras da classe 1 e 200 da classe 2

# Usando a variância ajustada de 0.16 para manter os dados separáveis

covariancia = [[0.16, 0],

               [0, 0.16]]


X1 = np.random.multivariate_normal([2, 2], covariancia, n_amostras)

X2 = np.random.multivariate_normal([4, 4], covariancia, n_amostras)


# Juntando os dados e adicionando o Bias (coluna de 1s)

X_dados = np.vstack((X1, X2))
```

```
bias = np.ones((X_dados.shape[0], 1))
X_com_bias = np.hstack((bias, X_dados))

# Criando os rótulos: 0 para Classe 1, 1 para Classe 2
y_dados = np.hstack((np.zeros(n_amostras), np.ones(n_amostras)))

# ==========================================
# 2. DIVISÃO EM TREINO E TESTE
# ==========================================
# Separando 70% para treinamento e 30% para teste de forma aleatória
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X_com_bias,
    y_dados,
    test_size=0.30,
    random_state=1 # Mantém a aleatoriedade reprodutível
)

# ==========================================
# 3. ALGORITMO DO PERCEPTRON
# ==========================================
def trainperceptron(X, y, eta=0.1, max_epochs=100):
    """Treina os pesos do Perceptron"""
    w = np.zeros(X.shape[1]) # Inicializa pesos com zero

    for epoca in range(max_epochs):
        erro_total = 0
        for i in range(X.shape[0]):
            u = np.dot(X[i], w)
            y_pred = 1 if u >= 0 else 0
```

```
        erro = y[i] - y_pred
        if erro != 0:
            w = w + eta * erro * X[i]
            erro_total += 1

    if erro_total == 0:
        break

  return w

def prever(X, w):
  """Aplica os pesos treinados para fazer previsões"""
  u = np.dot(X, w)
  return np.where(u >= 0, 1, 0)

# ==========================================
# 4. TREINAMENTO E AVALIAÇÃO
# ==========================================
# Usando o conjunto de treinamento para encontrar os pesos
w_treinado = trainperceptron(X_treino, y_treino)

# Avaliando o desempenho nos dois conjuntos
y_pred_treino = prever(X_treino, w_treinado)
y_pred_teste = prever(X_teste, w_treinado)

# Apresentando a acurácia e a matriz de confusão
print("==========================================")
print("=== RESULTADOS: CONJUNTO DE TREINAMENTO ===")
```

```
print("========================================")
print(f"Acurácia: {accuracy_score(y_treino, y_pred_treino) * 100:.2f}%")
print("Matriz de Confusão:")
print(confusion_matrix(y_treino, y_pred_treino))


print("\n========================================")
print("=== RESULTADOS: CONJUNTO DE TESTE ===")
print("========================================")
print(f"Acurácia: {accuracy_score(y_teste, y_pred_teste) * 100:.2f}%")
print("Matriz de Confusão:")
print(confusion_matrix(y_teste, y_pred_teste))
```

Resultados:

```
========================================
=== RESULTADOS: CONJUNTO DE TREINAMENTO ===
========================================
Acurácia: 100.00%
Matriz de Confusão:
[[148   0]
 [  0 132]]

========================================
=== RESULTADOS: CONJUNTO DE TESTE ===
========================================
Acurácia: 100.00%
Matriz de Confusão:
[[52  0]
 [ 0 68]]
```

Questão 3)

Código:

```
import numpy as np
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

# ==========================================
# 1. FUNÇÕES DO PERCEPTRON (Copiadas do Ex 2)
# ==========================================
def trainperceptron(X, y, eta=0.1, max_epochs=20):
    """Treina os pesos do Perceptron. Aumentei max_epochs por segurança em dados reais."""
    w = np.zeros(X.shape[1])

    for epoca in range(max_epochs):
        erro_total = 0
        for i in range(X.shape[0]):
            u = np.dot(X[i], w)
            y_pred = 1 if u >= 0 else 0

            erro = y[i] - y_pred
            if erro != 0:
                w = w + eta * erro * X[i]
                erro_total += 1

    return w

def prever(X, w):
    u = np.dot(X, w)
    return np.where(u >= 0, 1, 0)

# ==========================================
# 2. CARREGAMENTO E PREPARAÇÃO DOS DADOS IRIS
```

```
# ==========================================
print("Carregando base de dados Iris...")
iris = load_iris()
X_iris = iris.data
y_iris_original = iris.target

# Adicionando a coluna do Bias (coluna de 1s) nas 4 dimensões originais
bias = np.ones((X_iris.shape[0], 1))
X_com_bias = np.hstack((bias, X_iris))

# Ajustando os rótulos conforme o PDF:
# Espécie 1 (target 0) vira Classe 0. Espécies 2 e 3 (targets 1 e 2) viram Classe 1.
y_binario = np.where(y_iris_original == 0, 0, 1)

# ==========================================
# 3. LOOP DE 100 TREINAMENTOS
# ==========================================
# Listas para guardar a acurácia de cada uma das 100 rodadas
acuracias_treino = []
acuracias_teste = []

print("Iniciando loop de 100 treinamentos...")

for i in range(100):
    # O parâmetro stratify=y_binario garante os 70/30 para CADA classe
    X_treino, X_teste, y_treino, y_teste = train_test_split(
        X_com_bias,
        y_binario,
        test_size=0.30,
```

```
    stratify=y_binario
    )

    # Treina o modelo
    w_treinado = trainperceptron(X_treino, y_treino)

    # Faz previsões
    y_pred_treino = prever(X_treino, w_treinado)
    y_pred_teste = prever(X_teste, w_treinado)

    # Calcula e guarda as acurácias
    acuracias_treino.append(accuracy_score(y_treino, y_pred_treino))
    acuracias_teste.append(accuracy_score(y_teste, y_pred_teste))

# ==========================================
# 4. RESULTADOS ESTATÍSTICOS
# ==========================================
print("\n" + "="*50)
print("=== RESULTADOS APÓS 100 ITERAÇÕES ===")
print("="*50)

print("CONJUNTO DE TREINAMENTO:")
print(f"Acurácia Média: {np.mean(acuracias_treino) * 100:.2f}%")
print(f"Variância:     {np.var(acuracias_treino):.6f}")

print("\nCONJUNTO DE TESTE:")
print(f"Acurácia Média: {np.mean(acuracias_teste) * 100:.2f}%")
print(f"Variância:     {np.var(acuracias_teste):.6f}")
```

```
print("\n" + "="*50)

print("=== MATRIZ DE CONFUSÃO (Última iteração do loop) \n [[TN, FP],\n[FN, TP]
===")

print("="*50)

print("Treinamento:")

print(confusion_matrix(y_treino, y_pred_treino))

print("\nTeste:")

print(confusion_matrix(y_teste, y_pred_teste))
```

Resultados:

```
==================================================
=== RESULTADOS APÓS 100 ITERAÇÕES ===
==================================================
CONJUNTO DE TREINAMENTO:
Acurácia Média: 100.00%
Variância:      0.000000

CONJUNTO DE TESTE:
Acurácia Média: 99.93%
Variância:      0.000014

==================================================
=== MATRIZ DE CONFUSÃO (Última iteração do loop)
 [[TN, FP],
[FN, TP] ===
==================================================
Treinamento:
[[35  0]
 [ 0 70]]

Teste:
[[15  0]
 [ 0 30]]
```

Questão 4)

Código:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

# ==========================================
# 1. FUNÇÕES DO PERCEPTRON
# ==========================================
def trainperceptron(X, y, eta=0.1, max_epochs=1000):
    """Treina os pesos do Perceptron.
    O max_epochs vai ser o critério de parada principal aqui,
    já que a base Breast Cancer não é perfeitamente separável linearmente.
    """
    w = np.zeros(X.shape[1])

    for epoca in range(max_epochs):
        erro_total = 0
        for i in range(X.shape[0]):
            u = np.dot(X[i], w)
            y_pred = 1 if u >= 0 else 0

            erro = y[i] - y_pred
            if erro != 0:
                w = w + eta * erro * X[i]
                erro_total += 1

        if erro_total == 0:
```

```
        break

    return w

def prever(X, w):
    u = np.dot(X, w)
    return np.where(u >= 0, 1, 0)

# ==========================================
# 2. CARREGAMENTO E PREPARAÇÃO DOS DADOS BREAST CANCER
# ==========================================
print("Carregando base de dados Breast Cancer...")
cancer_data = load_breast_cancer()
X_cancer = cancer_data.data
y_cancer = cancer_data.target # Já vem em 0 (maligno) e 1 (benigno)

# Adicionando a coluna do Bias (coluna de 1s) nas 30 características da base
bias = np.ones((X_cancer.shape[0], 1))
X_com_bias = np.hstack((bias, X_cancer))

# ==========================================
# 3. LOOP DE 100 TREINAMENTOS
# ==========================================
acuracias_treino = []
acuracias_teste = []

print("Iniciando loop de 100 treinamentos (isso pode levar alguns segundos)...")

for i in range(100):
```

```
    # Divisão 70/30 garantindo a proporção de tumores benignos/malignos
    X_treino, X_teste, y_treino, y_teste = train_test_split(
        X_com_bias,
        y_cancer,
        test_size=0.30,
        stratify=y_cancer
    )

    # Treina o modelo
    w_treinado = trainperceptron(X_treino, y_treino)

    # Faz previsões
    y_pred_treino = prever(X_treino, w_treinado)
    y_pred_teste = prever(X_teste, w_treinado)

    # Calcula e guarda as acurácias
    acuracias_treino.append(accuracy_score(y_treino, y_pred_treino))
    acuracias_teste.append(accuracy_score(y_teste, y_pred_teste))

# ==========================================
# 4. RESULTADOS ESTATÍSTICOS
# ==========================================
print("\n" + "="*50)
print("=== RESULTADOS APÓS 100 ITERAÇÕES ===")
print("="*50)

print("CONJUNTO DE TREINAMENTO:")
print(f"Acurácia Média: {np.mean(acuracias_treino) * 100:.2f}%")
print(f"Variância:     {np.var(acuracias_treino):.6f}")
```

```
print("\nCONJUNTO DE TESTE:")
print(f"Acurácia Média: {np.mean(acuracias_teste) * 100:.2f}%")
print(f"Variância:     {np.var(acuracias_teste):.6f}")


print("\n" + "="*50)
print("=== MATRIZ DE CONFUSÃO (Última iteração do loop) ===")
print("="*50)
print("Treinamento:")
print(confusion_matrix(y_treino, y_pred_treino))
print("\nTeste:")
print(confusion_matrix(y_teste, y_pred_teste))
```

Resultados:

```
==================================================
=== RESULTADOS APÓS 100 ITERAÇÕES ===
==================================================
CONJUNTO DE TREINAMENTO:
Acurácia Média: 89.88%
Variância:      0.002030

CONJUNTO DE TESTE:
Acurácia Média: 89.50%
Variância:      0.002700

==================================================
=== MATRIZ DE CONFUSÃO (Última iteração do loop) ===
==================================================
Treinamento:
[[116  32]
 [  1 249]]

Teste:
[[ 52  12]
 [  2 105]]
```